

Developer Toolkit: Terminal, Git & Python Basics

Topics to be covered:

→ CLI — examples

→ Git commands

→ Basic Python Programming.

1 Navigation Commands

Purpose	Linux / PowerShell	CMD	Example
Show current folder	<code>pwd</code>	<code>cd</code>	<code>pwd</code>
List files	<code>ls</code>	<code>dir</code>	<code>ls</code>
Change directory	<code>cd folder</code>	<code>cd folder</code>	<code>cd project</code>
Go back	<code>cd ..</code>	<code>cd ..</code>	<code>cd ..</code>
Clear terminal	<code>clear</code>	<code>cls</code>	<code>cls</code>

Print working directory

Maria/Class 2 : `cd ..`

2 Folder Commands

Purpose	Linux / PowerShell	CMD	Example
Create folder	<code>mkdir test</code>	<code>mkdir test</code>	<code>mkdir demo</code>
Remove empty folder	<code>rmdir test</code>	<code>rmdir test</code>	<code>rmdir demo</code>

`mkdir folder_name`

3 File Commands

Purpose	Linux	PowerShell	CMD
Create file	<code>touch a.txt</code>	<code>ni a.txt</code>	<code>type nul > a.txt</code>
View file	<code>cat a.txt</code>	<code>cat a.txt</code>	<code>type a.txt</code>
Delete file	<code>rm a.txt</code>	<code>rm .txt</code>	<code>del a.txt</code>
Copy file	<code>cp a.txt b.txt</code>	<code>cp a.txt b.txt</code>	<code>copy a.txt b.txt</code>
Rename file	<code>mv a.txt b.txt</code>	<code>mv a.txt b.txt</code>	<code>rename a.txt b.txt</code>

ni a.py
ni a.txt
ni a.csv
ni a.xlsx
ni a.python
ni a.py
ni a.py
ni a.py

4 Python Commands

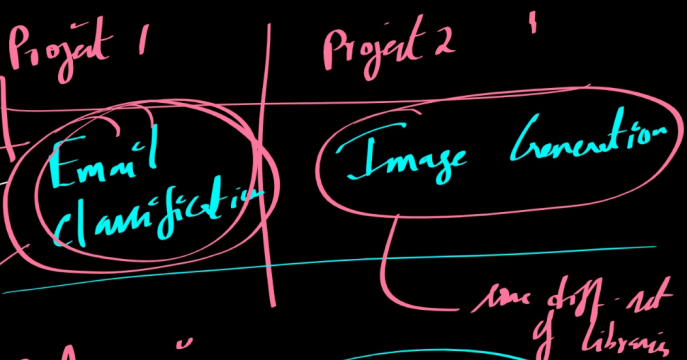
Purpose	Command
Check version	<code>python --version</code>
Open Python shell	<code>python</code>
Run Python file	<code>python app.py</code>
Install package	<code>pip install numpy</code>
Show packages	<code>pip list</code>
Freeze requirements	<code>pip freeze</code>

`python --version`

`python filename.py`

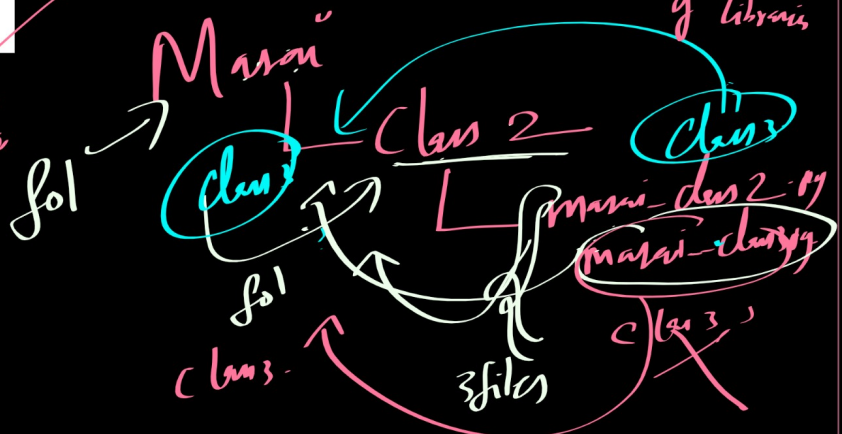
5 Virtual Environment Commands

Purpose	Command
Create venv	<code>python -m venv env</code>
Activate venv	<code>env\Scripts\activate</code>
Deactivate venv	<code>deactivate</code>



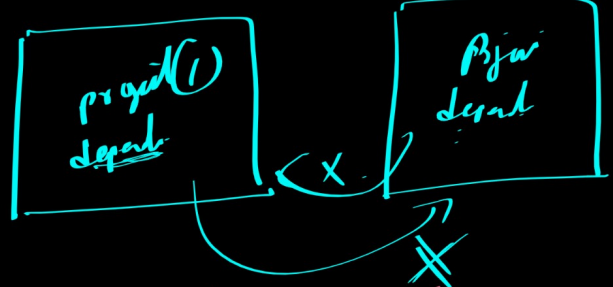
1

Some not 4 libraries

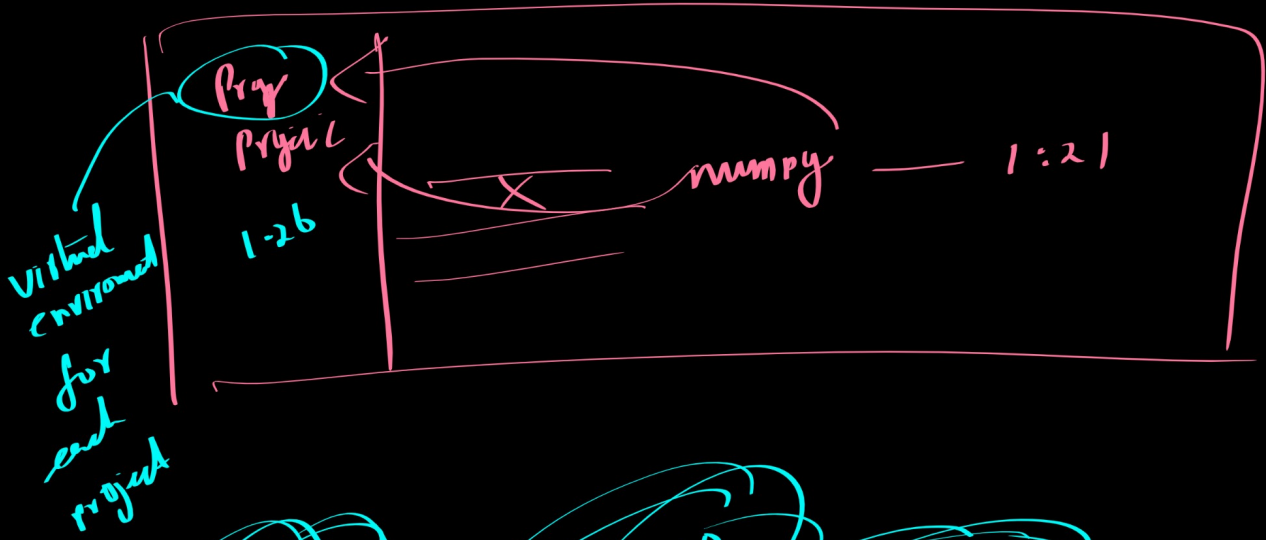
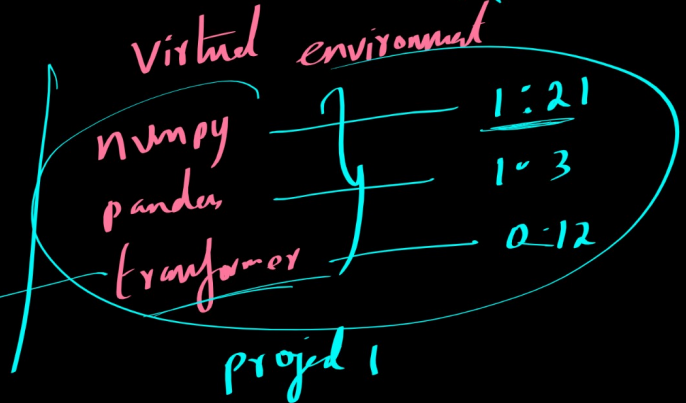


6 Git Commands (Very Important)

Purpose	Command
Initialize git	<code>git init</code>
Clone repo	<code>git clone <url></code>
Check status	<code>git status</code>
Add files	<code>git add .</code>
Commit	<code>git commit -m "message"</code>
Push code	<code>git push</code>
Pull latest code	<code>git pull</code>



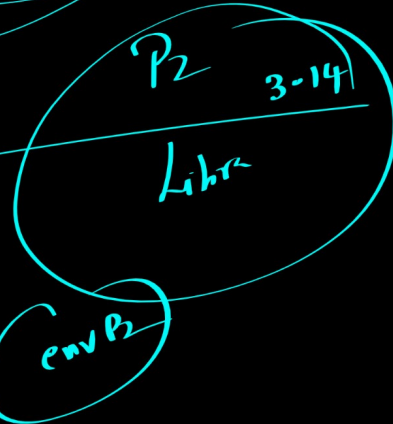
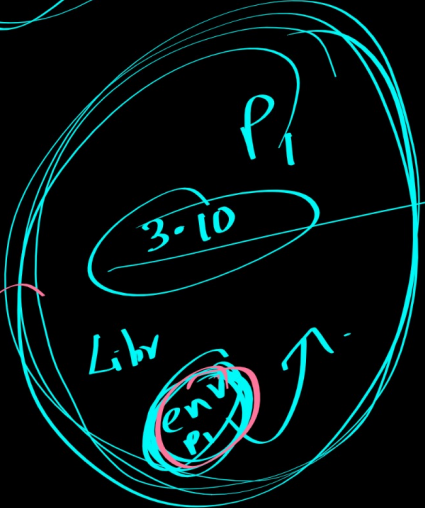
{ numpy 1.26
 version
 { tensorflow 0.8



python 3.14

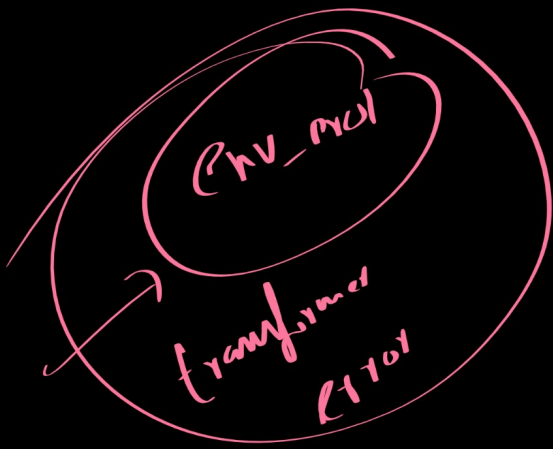
python 3.10

python 3.9



dependencies

Pip



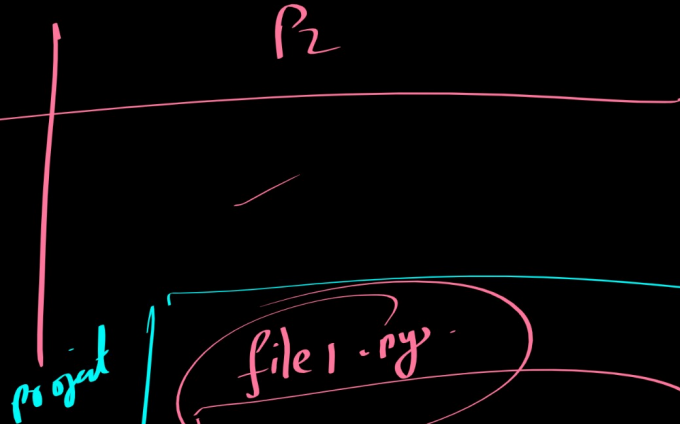
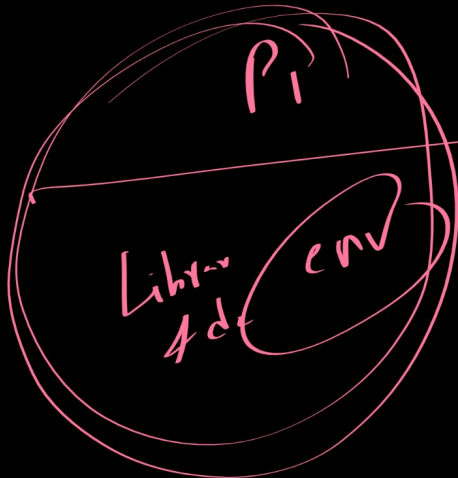
python -m venv env
env - act
3.14

Python 3.14

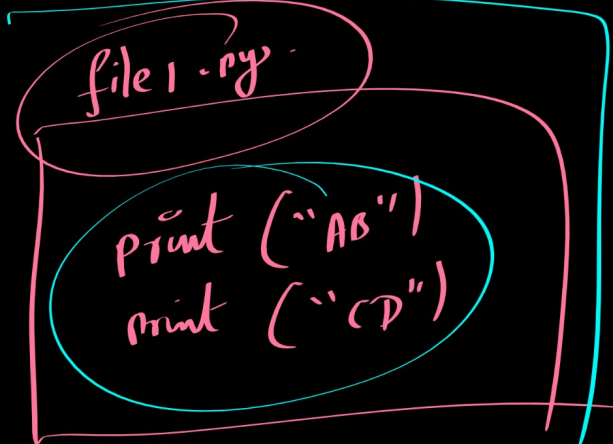
python 3.10

python3.10

-m venv env310
env310 - activate
python - venv
3.10



Git

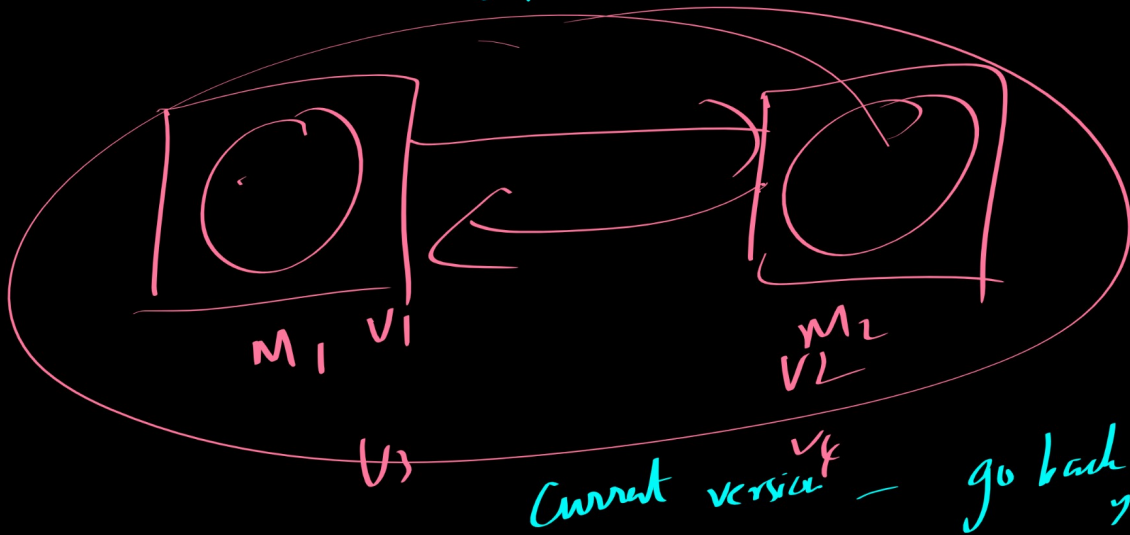


print(a)
print(c)
print(10)
print(20)

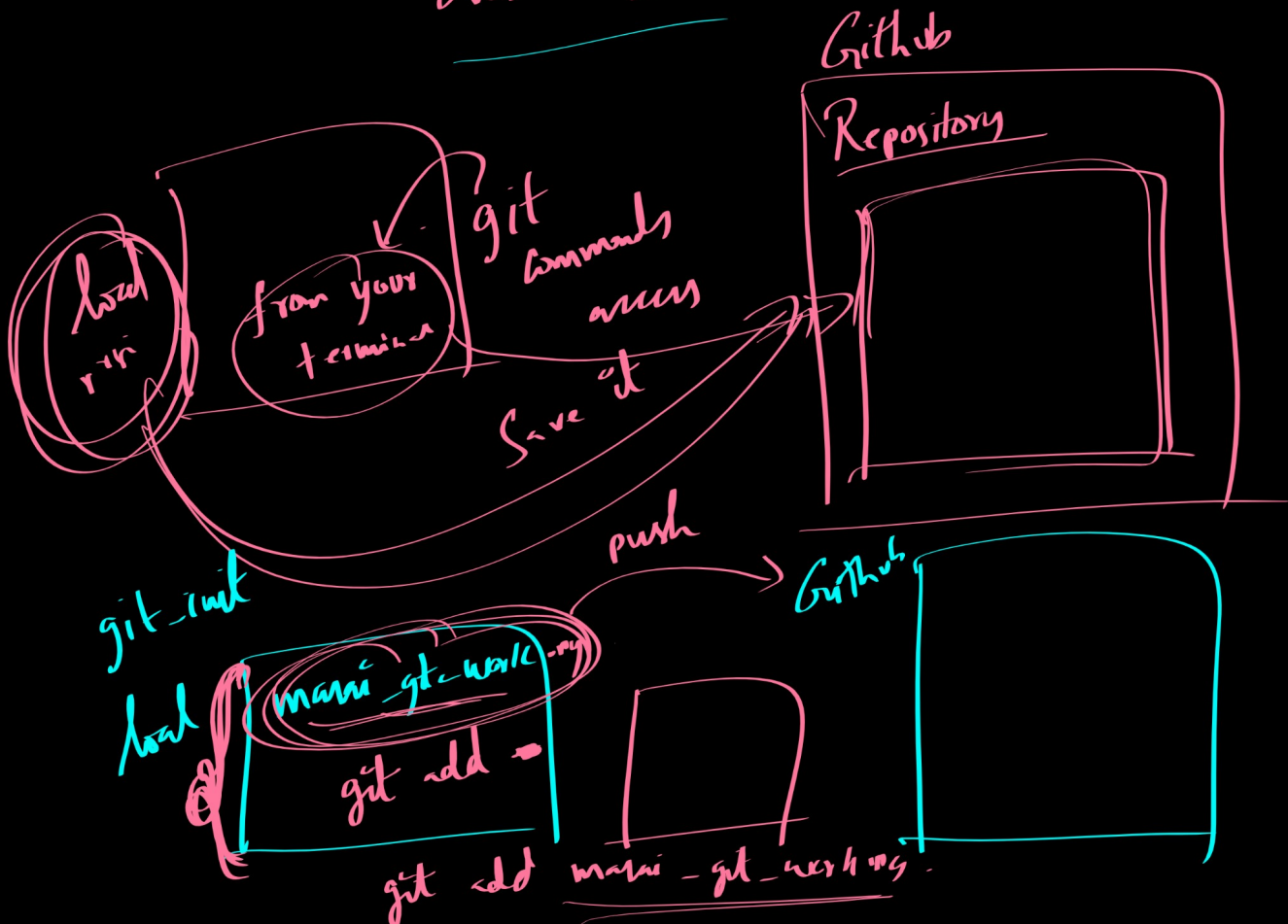
file1_v2.py

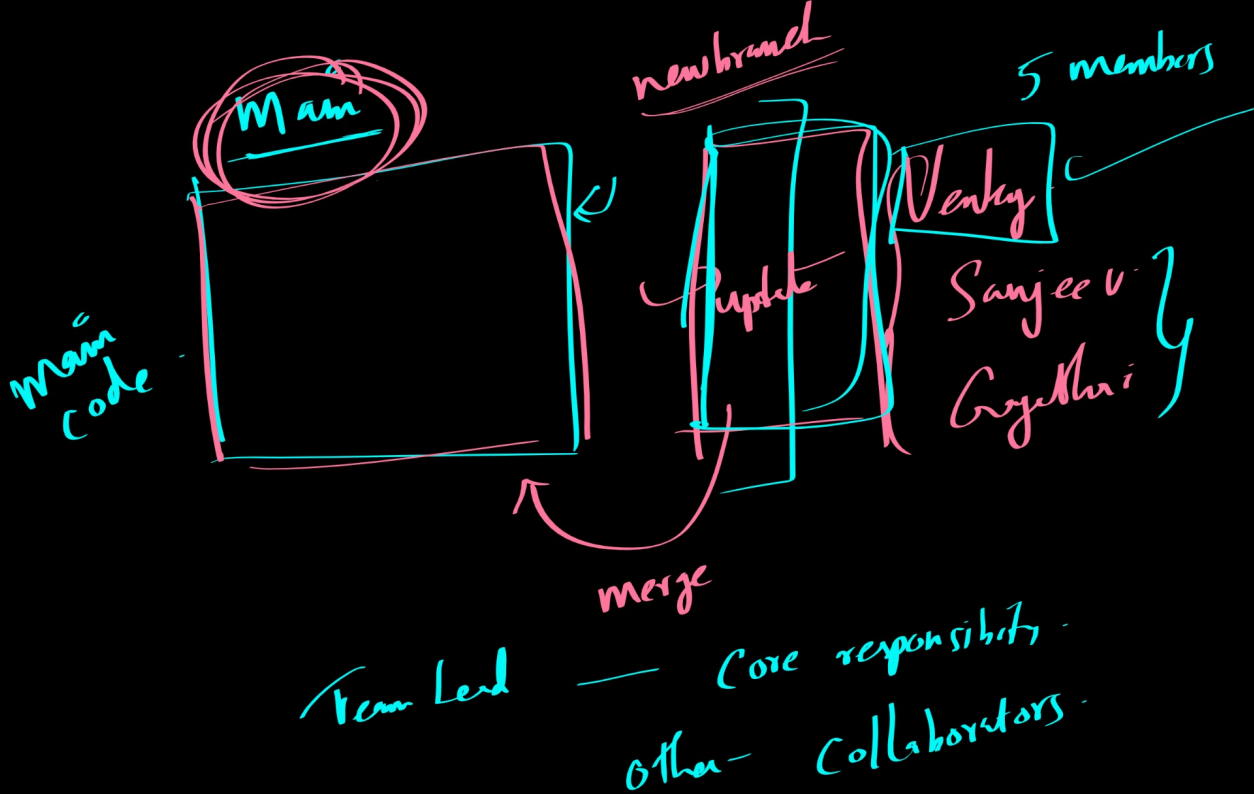
file - vs - py

v4 v7

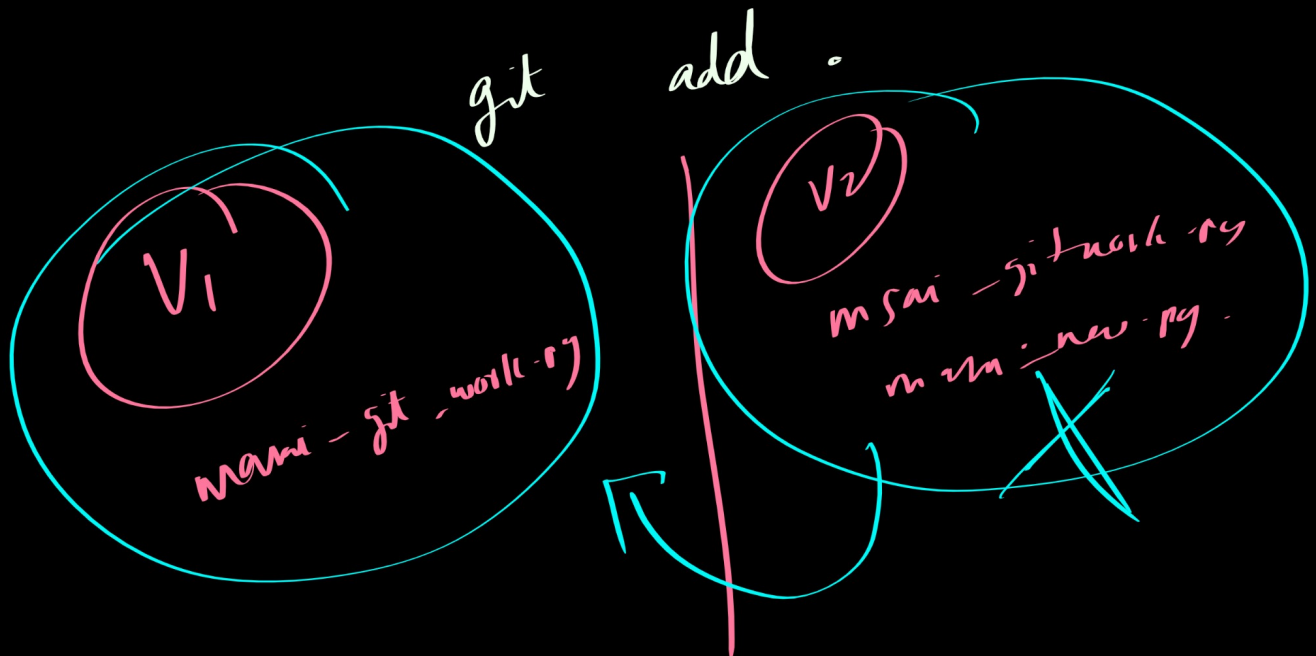


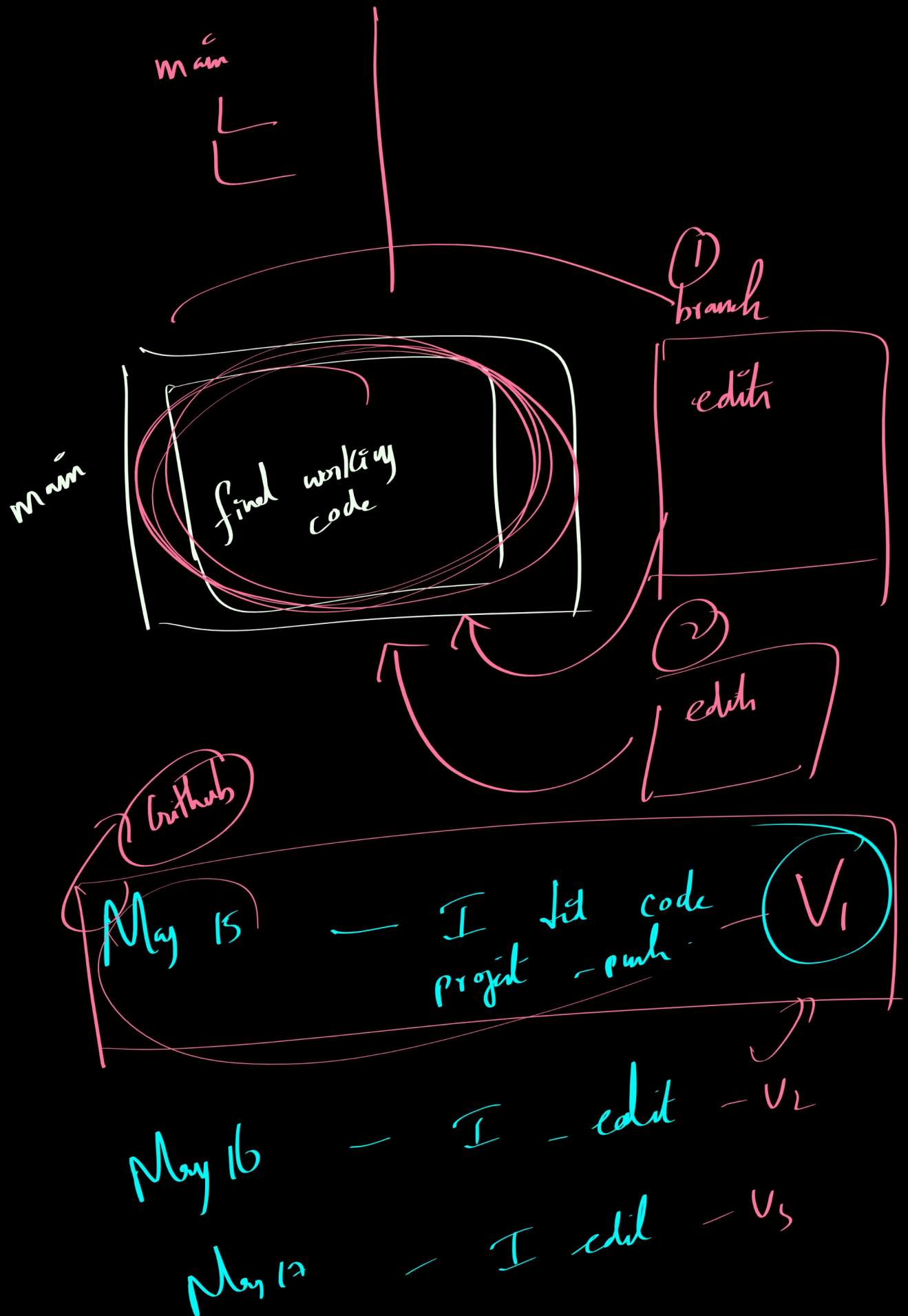
Version control system that tracks changes in your project over time.





git clone <https://github.com/venkateshelangovan/project1-masai.git>





May 15

I want to
go back to
my intro ver.

Issue

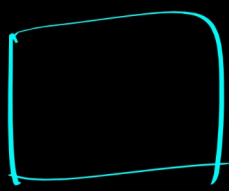
July 20

I do edit

V₁₀₀

<https://git-scm.com/install/windows>

Python Programming -



work & calculations/
decisions / build
something

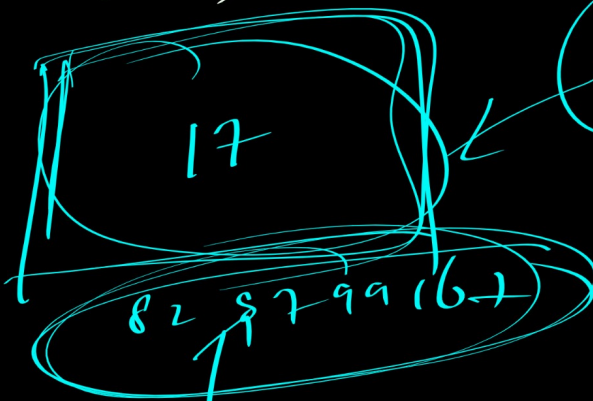
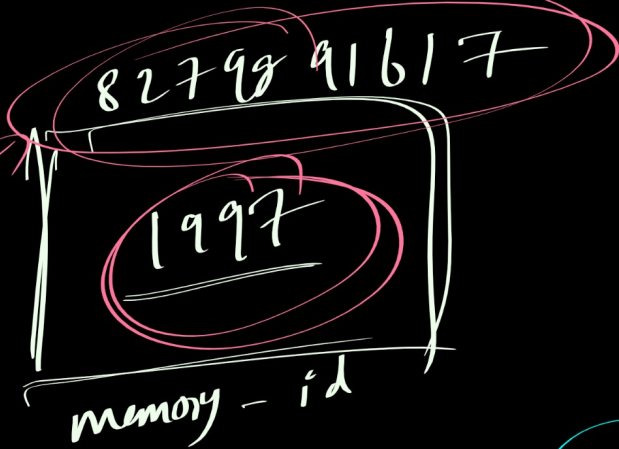
Store some data

Variables:

Value =

8.9

labelled storage box
in memory

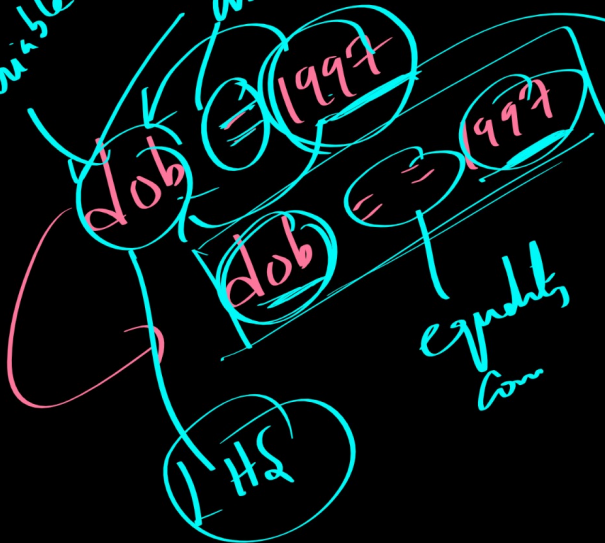


dob

dob

variable

unique



equals
com

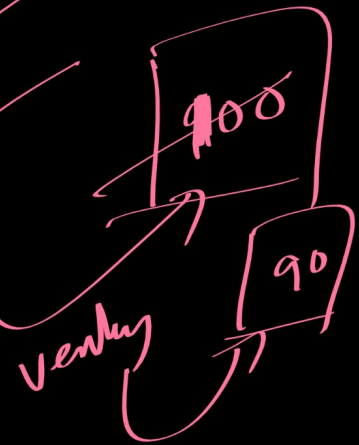
not
Value unique

Variable Naming Conventions

venky = 100 ✓

venky = 90 ✓

print(venky) = 90 ✓

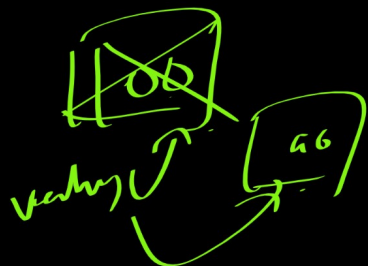


- L1 V = 10 ✓
- L2 V = 8 ✓
- L3 V = 6 ✓
- L4 V = 5 ✓
- L5 print(v) ✓ → (5)

venky = 100 ✓
 print("Venky is 100 here: ", venky) ✓
 venky = 90 ✓
 print("Venky is 90 here: ", venky) ✓

100

90



int
 100
 8
 2
 12
 256
 512

float
 0.5
 11.2
 12.998
 11.999

256.0 — read it as float

"a", "b", "c", ..., "z"

"Vish", "IIIT Hyd"

Variable
name

name_2 = "name_4"

name_2 — value

~~name_2 = name_4~~

int, str, float

```
if cgpa > 9:  
    grade = S  
else:  
    grade = "Fail"
```

cgpa = 8.5

if cgpa > 9:

print("Grade : S")

else:

print("Grade : F")

256

256

cgpa = 8.5

if cgpa > 9:

print("Grade : S")

print(100)

print(1000)

else:

print("Grade : F")

print(-100)

print(-1000)

print(10000)

Fail, Truth

Grade : F
-100
-1000

> 9

elif > 8

> 7

> 6

> 5

elif

if :

else :

Print (10000)

else if

if cgpa > 9 :

elif cgpa > 8 :

elif cgpa > 7 :

else :

cgpa = 8.99

if cgpa > 9:

print("S")

elif cgpa > 7:

print("B")

elif cgpa > 8:

print("A")

else:

print("C")

B

B

A